

Gdańsk, Poland, 14 July 2026

Dear Editors,

Please consider my manuscript, "A Comparative Analysis of Relational Database Access-Layer Performance in Express.js for PostgreSQL and MySQL," for publication in *Information and Software Technology* as a Research Paper.

Node.js/Express services reach relational databases through a spectrum of access layers: native drivers, query builders, and object-relational mappers (ORMs). This choice is made on nearly every non-trivial back-end project, yet in practice it is guided by vendor benchmarks that are fragmented: each is published by the maintainers of one of the compared products, almost all target PostgreSQL exclusively, and each reports a single metric family (throughput or latency, rarely the p99 tail that dominates perceived performance under load). The peer-reviewed literature is sparser still: the closest prior study compared ORM overhead across eight frameworks but explicitly excluded JavaScript, and two 2025 studies of Node.js/TypeScript access layers are each limited to PostgreSQL and to at most three ORMs. In the sources I searched through July 2026, I did not identify a study combining broad taxonomy coverage, both major open-source engines, vendor independence, and joint throughput/tail reporting.

I address that gap with a benchmark combining all four properties. An identical Express application is served through nine idiomatic access layers (the native drivers pg and mysql2, the query builder Knex, and six ORMs: Drizzle, Prisma, Sequelize, TypeORM, Objection, and MikroORM) plus two tuned native baselines, over five selected access patterns spanning common CRUD operations, against both PostgreSQL and MySQL, reporting throughput and p99 tail latency jointly across all 90 measured configurations. Because within each replicate every layer is driven by the identical request stream, the design is a randomized block, and layers are compared with paired tests on their per-replicate throughput ratios; each configuration is measured over 25 independent replicates (a freshly booted server process per replicate, cell order randomized), and before any measurement an automated cross-check verifies that every layer returns byte-identical responses. Inserts are measured under each engine's default durability from a physically rebuilt database state. A same-SQL control bounds the largest effect observed, a native-relative spread of 6.5x between the native driver and the heaviest ORM on a deep, nested fetch: on identical SQL (byte-identical statements confirmed from server-side logs, wire protocols disclosed) that spread narrows to at most 1.5x, so the idiomatic deficit lies upstream of raw SQL execution, in each library's query-generation and result-construction path, which the control changes as a bundle rather than isolating; a fan-out sweep shows it grows with the size of the fetched graph. Further controls corroborate the main results: a coordinated-omission-corrected open-loop validation of the tail-latency findings, an equal-CPU-budget run and combined application-plus-database CPU accounting, a pool-size sensitivity sweep, a single-host resource-isolation check, a transactional multi-statement write, and a symmetric relaxed-durability secondary showing the engine contrast is not a durability artifact. The complete replication package (harness, deterministic seed, all adapters, raw measurements, and the scripts that generate every table in the paper) is public on GitHub and permanently archived on Zenodo (DOI: 10.5281/zenodo.21359919); a numbered online supplement accompanies the manuscript.

The manuscript fits *Information and Software Technology* directly: it is a controlled empirical study of a decision that essentially every Node.js/Express project makes—which access layer to adopt—aimed at improving that practice with evidence rather than vendor marketing. It is reported to the standard the journal expects of empirical software engineering work: controlled factors, independent replicates, paired nonparametric significance and equivalence testing with effect sizes, an explicit threats-to-validity analysis, and an open, reusable replication package.

The work is original, has not been published previously, and is not under consideration else-

where. I declare no competing interests and received no funding for this research. A declaration of generative-AI use in the manuscript-preparation process is included in the manuscript in accordance with Elsevier policy.

I look forward to your editorial decision.

Sincerely,

Mateusz Miotk Faculty of Mathematics, Physics and Informatics, University of Gdańsk, Poland
mateusz.miotk@ug.edu.pl (corresponding author)